



프로그램 언어

10주차



인하대학교



터틀 그래픽 (Turtle모듈)

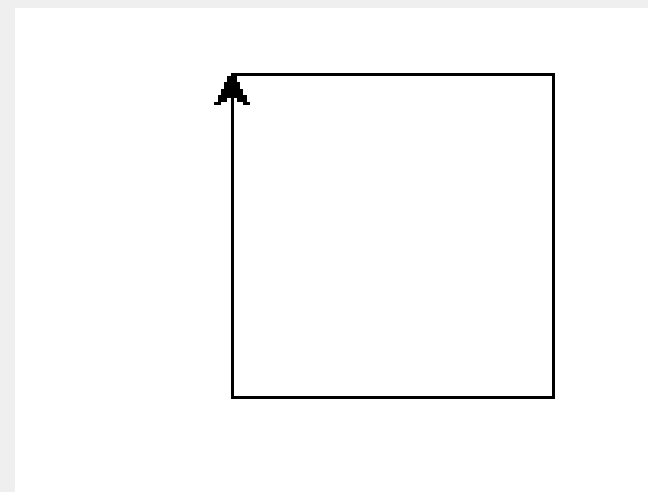
터틀 그래픽(Turtle graphic)

- 터틀 그래픽은 화면에서 거북이를 이용하여 그림을 그리는 기능.
- 터틀그래픽은 파이썬의 기본모듈
- 사용법 : `import turtle`
- 터틀그래픽을 시작하면
 - 창을 만들고
 - 한가운데(0,0)에 거북이를 오른쪽으로 향하도록 세워놓고
 - 명령대기
- 모양을 따로 지정하지 않으면 화살촉모양

```
import turtle
turtle.shape("turtle")
turtle.forward(100)
turtle.right(90)
turtle.forward(100)
```

사각형 그리기

```
import turtle as t  
t.shape("turtle")  
t.forward(100)  
t.right(90)  
t.forward(100)  
t.right(90)  
t.forward(100)  
t.right(90)  
t.forward(100)
```



정삼각형을 그려보자

정삼각형



인하대학교

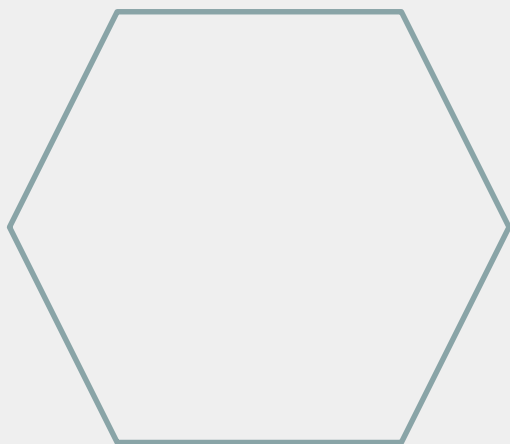
```
import turtle as t  
t.shape("turtle")  
t.forward(100)  
t.left(120)  
t.forward(100)  
t.left(120)  
t.forward(100)
```

실습



인하대학교

- 화살표를 움직여 6각형을 그려보자 (회전각도는 몇도?)



```
import turtle as t
t.shape("turtle")

for i in range(6):
    t.forward(100)
    t.right(360/6)
```

모듈(module)



인하대학교

The screenshot shows the Python.org website. At the top, there's a navigation bar with links like 'About', 'Downloads', 'Documentation', 'Community', 'Success Stories', 'News', and 'Events'. Below this, a large banner encourages users to 'Browse the docs online or download a copy of your own.' and provides a 'Python Docs' button. Further down, there are sections for 'Beginner', 'Moderate', and 'Advanced' resources. In the 'Python 3.x Resources' section, the 'Module Index' link is highlighted with a red box.

+ tkinter	Interface to Tcl/Tk for graphical user interfaces
token	Constants representing terminal nodes of the parse tree.
tokenize	Lexical scanner for Python source code.
trace	Trace or track Python statement execution.
traceback	Print or retrieve a stack traceback.
tracemalloc	Trace memory allocations.
tty (Unix)	Utility functions that perform common terminal control operations.
turtle	An educational framework for simple graphics applications
turtledemo	A viewer for example turtle scripts
types	Names for built-in types.
typing	Support for type hints (see :pep:`484`).
u	
unicodedata	Access the Unicode Database.
+ unittest	Unit testing framework for Python.
+ urllib	
uu	Deprecated: Encode and decode files in uuencode format.
uuid	UUID objects (universally unique identifiers) according to RFC 4122

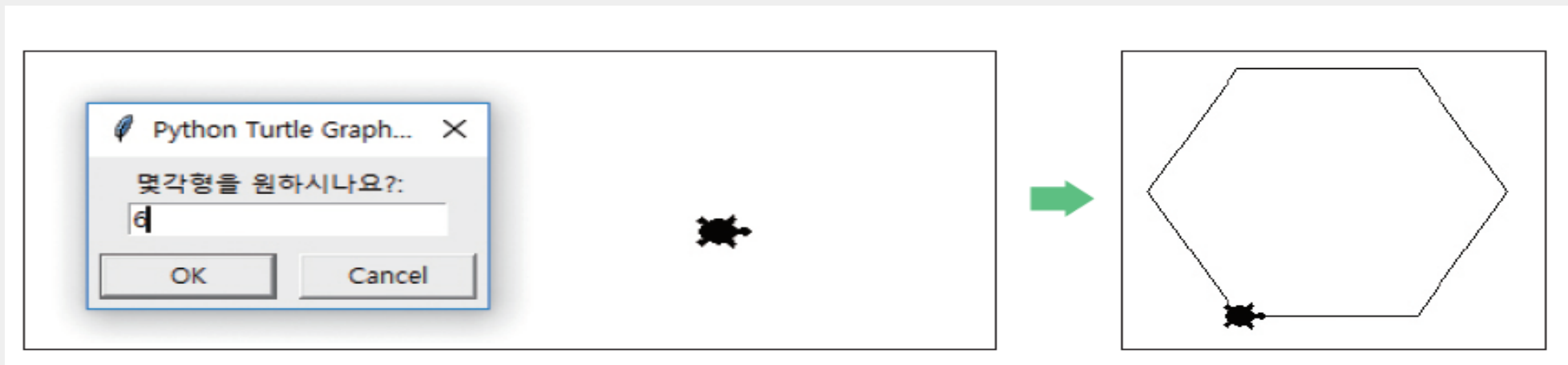
조작함수

함수	인수	설명
forward()	픽셀값	거북이를 지정된 거리만큼 앞으로 이동한다.
backward()	픽셀값	거북이를 지정된 거리만큼 뒤로 이동한다.
right()	각도	거북이를 시계방향으로 회전시킨다.
left()	각도	거북이를 반시계방향으로 회전시킨다.
penup()	None	거북이의 펜을 올린다. 그림이 그려지지 않는다.
pendown()	None	거북이의 펜을 내린다. 그림이 그려진다.
up()	None	거북이의 펜을 올린다. 그림이 그려지지 않는다.
down()	None	거북이의 펜을 내린다. 그림이 그려진다.
color()	색상 이름	거북이 펜의 색상을 변경한다.
fillcolor()	색상 이름	다각형을 채우는 색상을 변경한다.
heading()	None	현재의 방향을 반환한다.
position()	None	현재 위치를 반환한다.
goto()	x, y	거북이를 (x, y) 위치로 이동시킨다.
begin_fill()	None	채워진 다각형을 시작한다.
end_fill()	None	채워진 다각형을 닫는다.
dot()	None	현재 위치에 점을 남긴다.
stamp()	None	현재 위치에 거북이 모양을 남긴다.
shape()	모양 이름	거북이의 모양을 'arrow', 'classic', 'turtle', 'circle' 중의 하나로 변경한다.

- circle(반지름) : 현재위치에서 원을 그린다. t.circle(50)
- speed(속도) : 거북이 속도 0~10
- pensize(), width() : 펜굵기
- bgcolor('색이름') : 화면의 배경색
- textinput('윈도우이름', '텍스트') : 사용자로부터 입력받기

N각형 그리기

- 터틀 그래픽에서 사용자로부터 정수 n 을 받아서 n -각형을 그리는 프로그램을 작성해 보자.



```
import turtle as t
```

```
t.shape("turtle")
```

```
s = t.textinput( " 도형그리기", "몇각형을 원하시나요?:")
```

```
n = int(s)
```

```
for i in range(n):
```

```
    t.forward(100)
```

```
    t.left(360/n)
```

실습



인하대학교

(0,100)

길이:100픽셀

(0,0)

```
import turtle as t
```

```
t.shape("turtle")
```

```
t.goto(0, 0)
```

```
t.forward(100)
```

```
t.up()
```

```
t.goto(0, 100)
```

```
t.down()
```

```
t.forward(100)
```

실습

- 터틀 그래픽을 사용하여 반지름이 100픽셀인 3개의 원을 그리는 프로그램.(간격은 30픽셀)

```
import turtle as t  
t.shape("turtle")
```

```
radius = 100
```

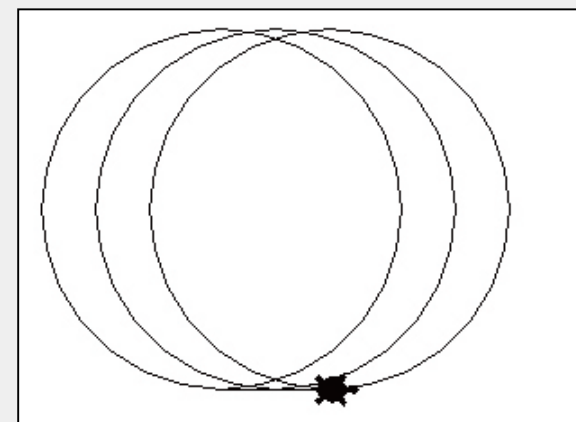
```
t.circle(radius) # 반지름이 100인 원이 그려 진다.
```

```
t.fd(30)
```

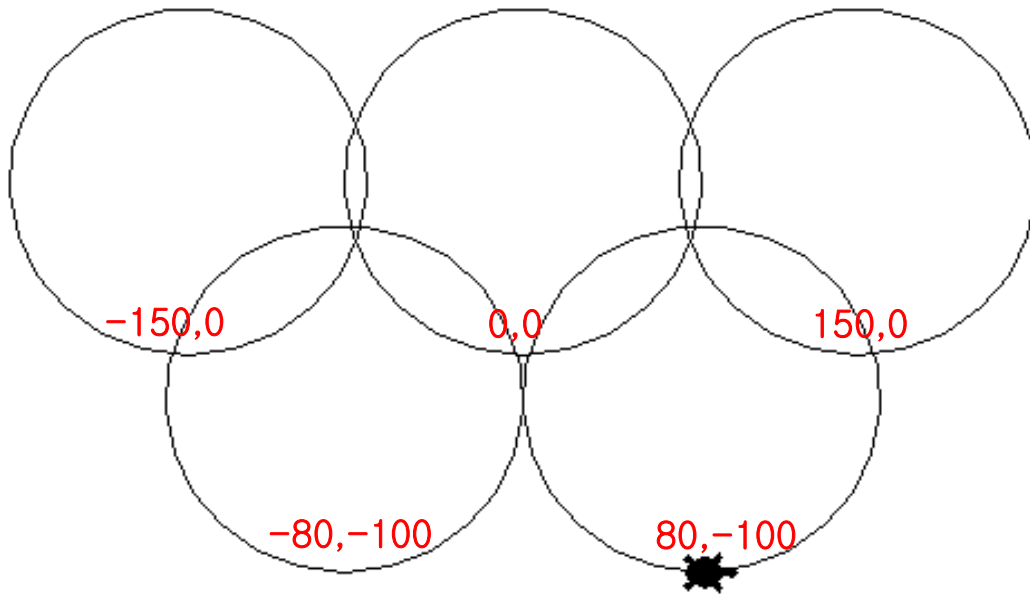
```
t.circle(radius) # 반지름이 100인 원이 그려 진다.
```

```
t.fd(30)
```

```
t.circle(radius) # 반지름이 100인 원이 그려 진다.
```



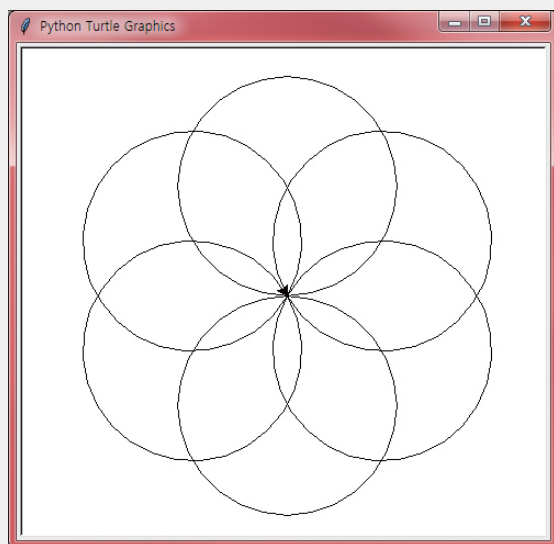
실습 : 오륜기



```
import turtle  
t.shape("turtle")
```

```
t.up()  
t.goto(-150, 0)  
t.down()  
t.circle(80)
```

```
t.up()  
t.goto(0, 0)  
t.down()  
t.circle(80)  
t.up()  
t.goto(150, 0)  
t.down()  
t.circle(80)  
t.up()  
t.goto(-80, -100)  
t.down()  
t.circle(80)  
t.up()  
t.goto(80, -100)  
t.down()  
t.circle(80)
```



```
import turtle as t
```

```
for count in range(6):
```

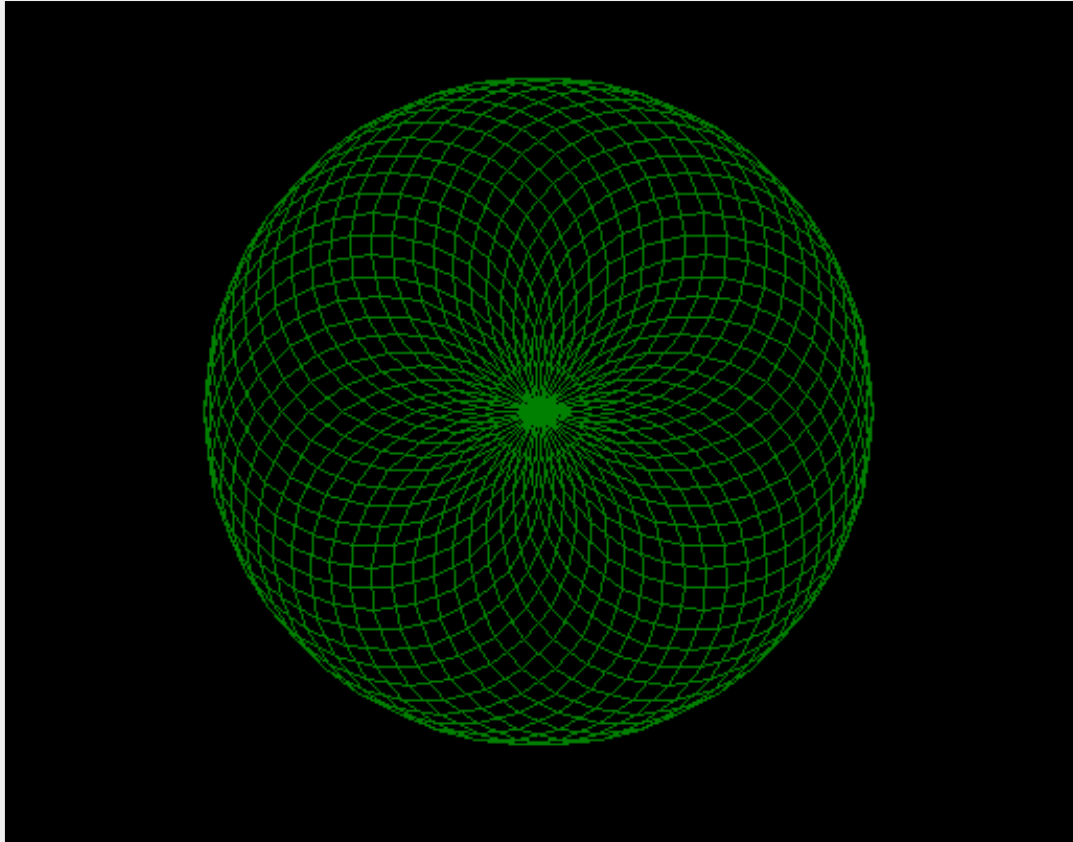
```
    t.circle(100)
```

```
    t.left(360/6)
```

실습



인하대학교



```
import turtle as t
```

```
n=50
```

```
t.bgcolor('black')
```

```
t.color('green')
```

```
t.speed(0)
```

```
for i in range(n):
```

```
    t.circle(80)
```

```
    t.left(360/n)
```


색깔 채우기

```
import turtle as t

t.shape("turtle")

t.fillcolor("yellow") # 채우기 색상을 설정한다.

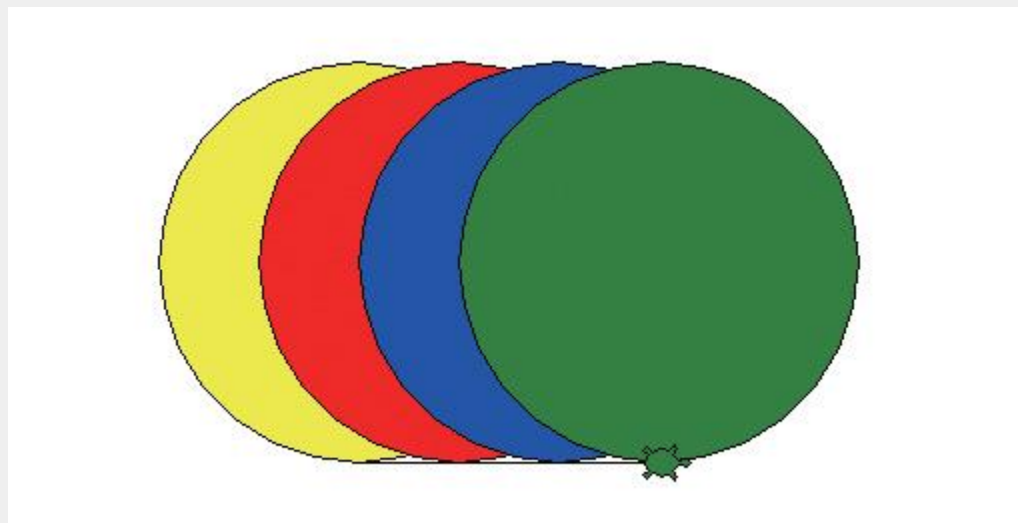
t.begin_fill() # 채우기를 시작한다.

t.circle(100) # 속이 채워진 원이 그려진다.

t.end_fill() # 채우기를 종료한다.
```

실습

- 리스트에 색상을 문자열로 저장하였다가 하나씩 꺼내서 거북이의 채우기 색상으로 설정하고 원을 그려 보자.



```
import turtle as t
t.shape("turtle")

color_list = [ "yellow", "red", "blue", "green" ]

t.fillcolor(color_list[0])
t.begin_fill()
t.circle(100)
t.end_fill()

t.forward(50)
t.fillcolor(color_list[1])
t.begin_fill()
t.circle(100)
t.end_fill()
...
```

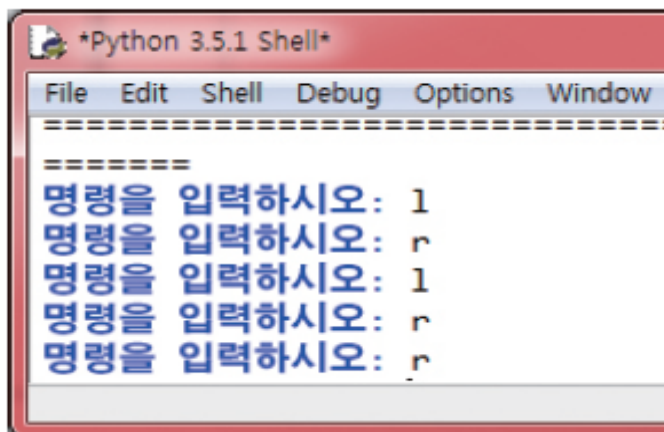
```
import turtle as t
t.shape("turtle")

color_list = [ "yellow", "red", "blue", "green" ]

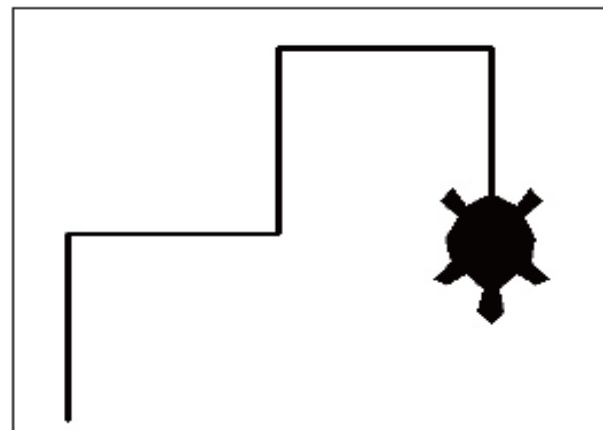
for i in color_list:
    t.begin_fill()
    t.fillcolor(i)
    t.circle(100)
    t.forward(30)
    t.end_fill()
```

거북이 제어하기

- 파이썬 셸에서
- 사용자가 “l”을 입력하면 거북이가 왼쪽으로 100픽셀 이동하고
- “r”을 입력하면 거북이가 오른쪽으로 100픽셀 이동하는 프로그램.
- 사용자가 “q”를 입력하면 종료



```
*Python 3.5.1 Shell*
File Edit Shell Debug Options Window
=====
명령을 입력하시오: l
명령을 입력하시오: r
명령을 입력하시오: l
명령을 입력하시오: r
명령을 입력하시오: r
```



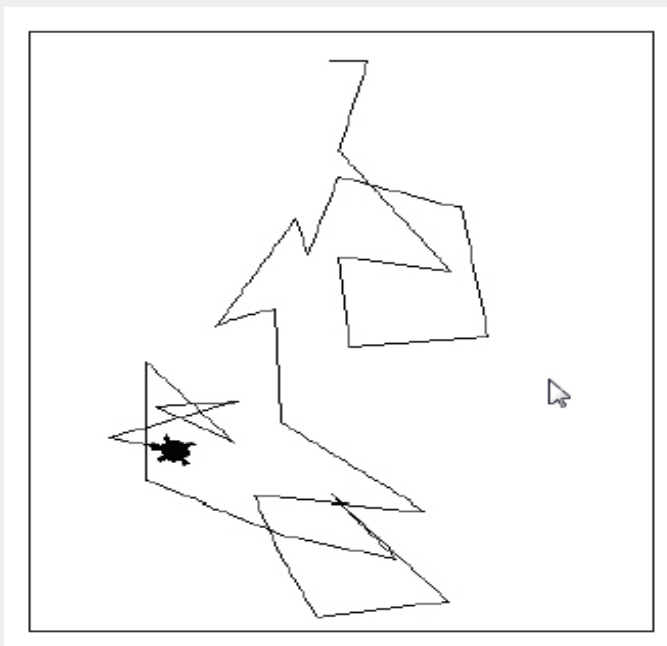
```
import turtle as t

t.shape("turtle")

while True:
    command = input("명령을 입력하시오: ")
    if command == "l":
        t.left(90)
        t.forward(100)
    if command == "r":
        t.right(90)
        t.forward(100)
    if command == "q":
        break
```

거북이 랜덤 워크

- 원도우 상에서 거북이가 술에 취한 것처럼 랜덤하게 움직이게 하여 보자.

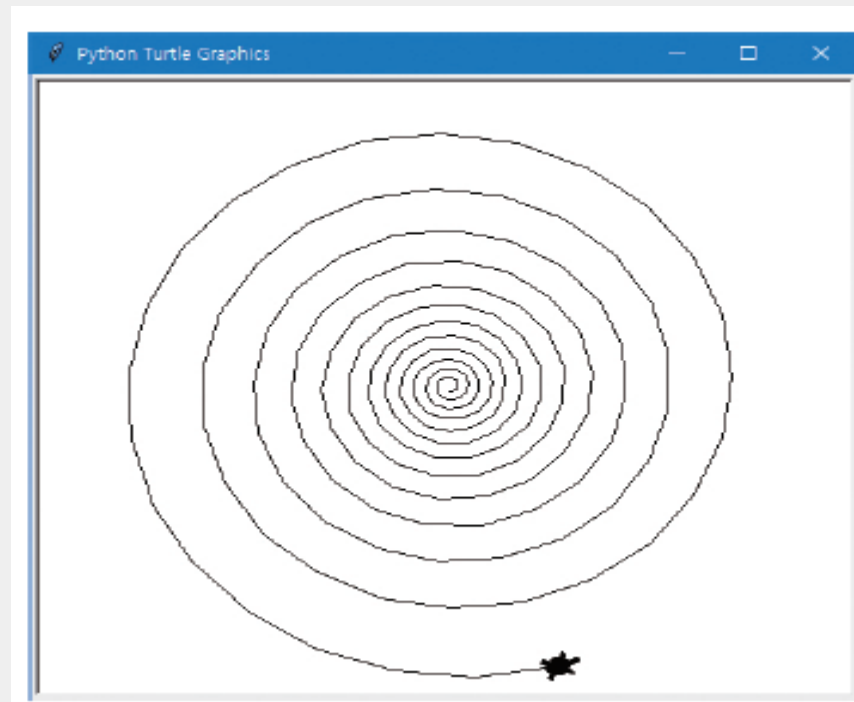


```
import turtle as t
import random as r
t.shape("turtle")

for i in range(30):
    length = r.randint(1, 100)
    t.forward(length)
    angle = r.randint(-180, 180)
    t.right(angle)
```

스파이럴1

- 반복문을 터틀 그래픽과 결합하면 상당히 복잡한 형상을 쉽게 그릴 수 있다.




```
import turtle as t
```

```
t.shape("turtle")
```

```
for i in range(200):
```

```
    t.forward(2+i/4)
```

```
    t.left(30-i/12)
```

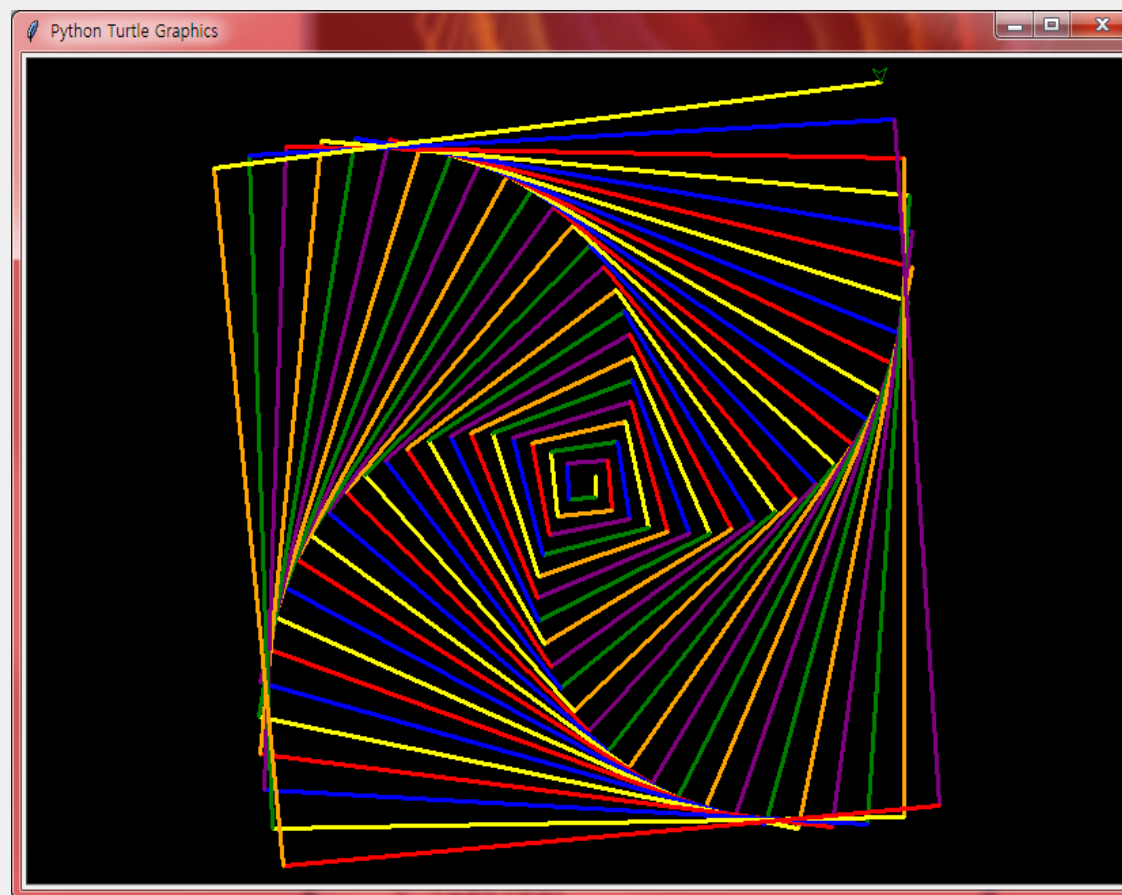
```
# 반복에 따라 조금씩 증가시킨다.
```

```
# 반복에 따라 조금씩 감소시킨다.
```

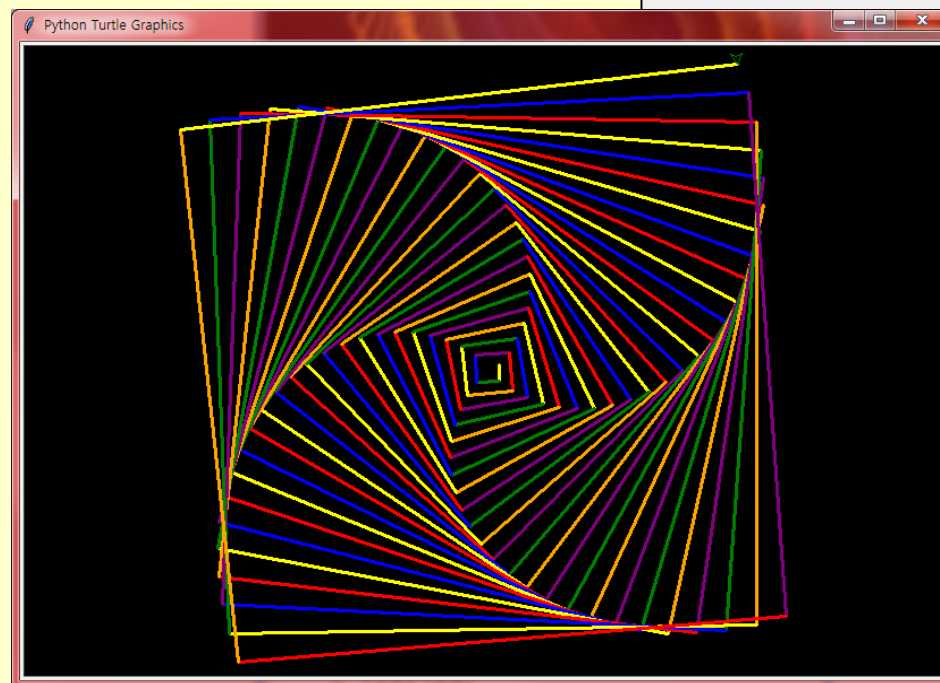
스파이럴2



인하대학교

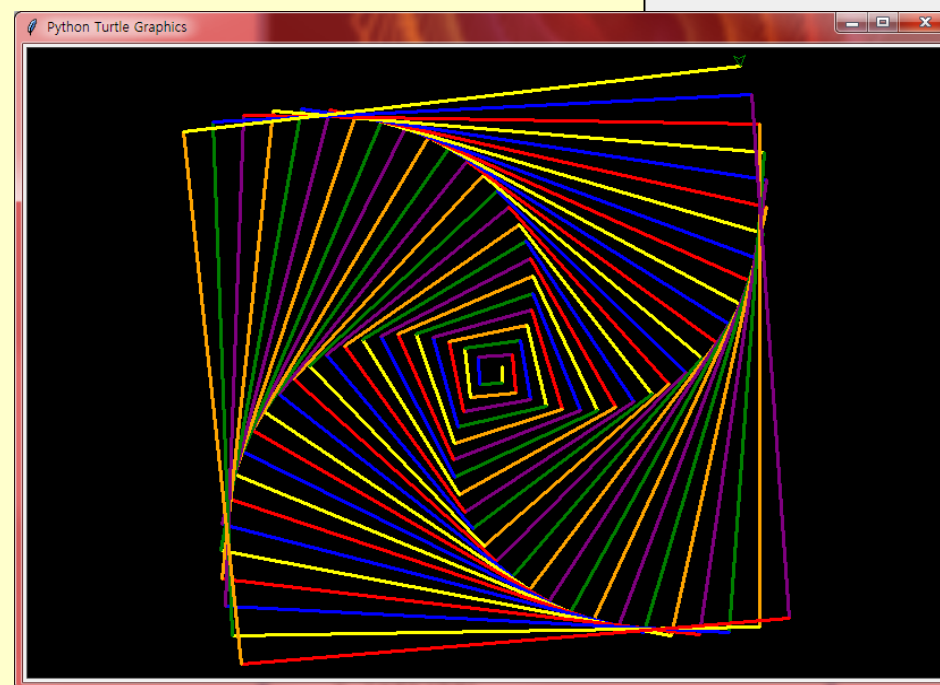


```
import turtle as t
t.shape('turtle')
color_list=['yellow','green','blue','purple','red','orange']
t.speed(0)
t.width(3)
t.bgcolor('black')
length=10
while length<500:
    for i in color_list:
        t.color(i)
        t.right(89)
        t.forward(length)
        length+=5
```



```
import turtle as t
colors = ['red','purple','blue','green','yellow','orange']
t.bgcolor("black")
t.speed(0)
t.width(3)
```

```
length = 10
count=1
while length < 500:
    t.forward(length)
    t.pencolor(colors[count%6])
    t.right (89)
    length = length+5
    count=count+1
```





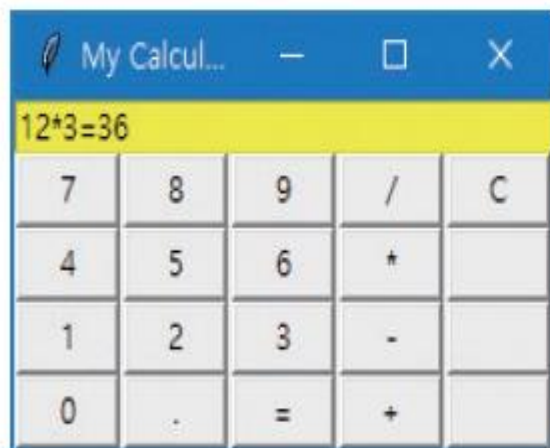
tkinter

tkinter

- Tkinter는 파이썬에서 그래픽 사용자 인터페이스(GUI: graphical user interface)를 개발할 때 필요한 모듈
- tkinter는 파이썬 설치시 기본으로 포함되는 그래픽 모듈
- 버튼, 레이블, 기타요소들을 이용하여 사용자와 상호작용하는 프로그램 작성가능



GUI 모드와 텍스트 모드의 비교



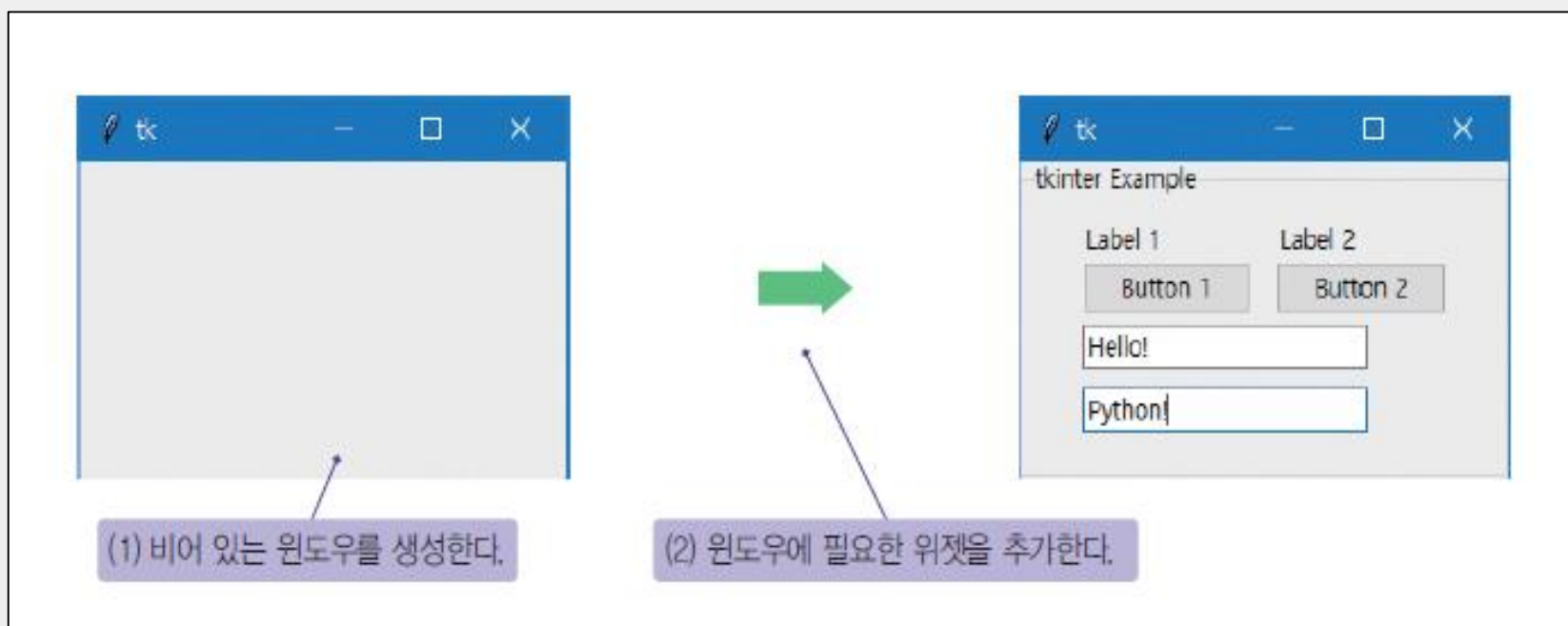
첫 번째 수를 입력하시오:45

두 번째 수를 입력하시오:6

연산의 종류를 입력하시오(+, -, *, /):-
연산의 결과는 39 입니다

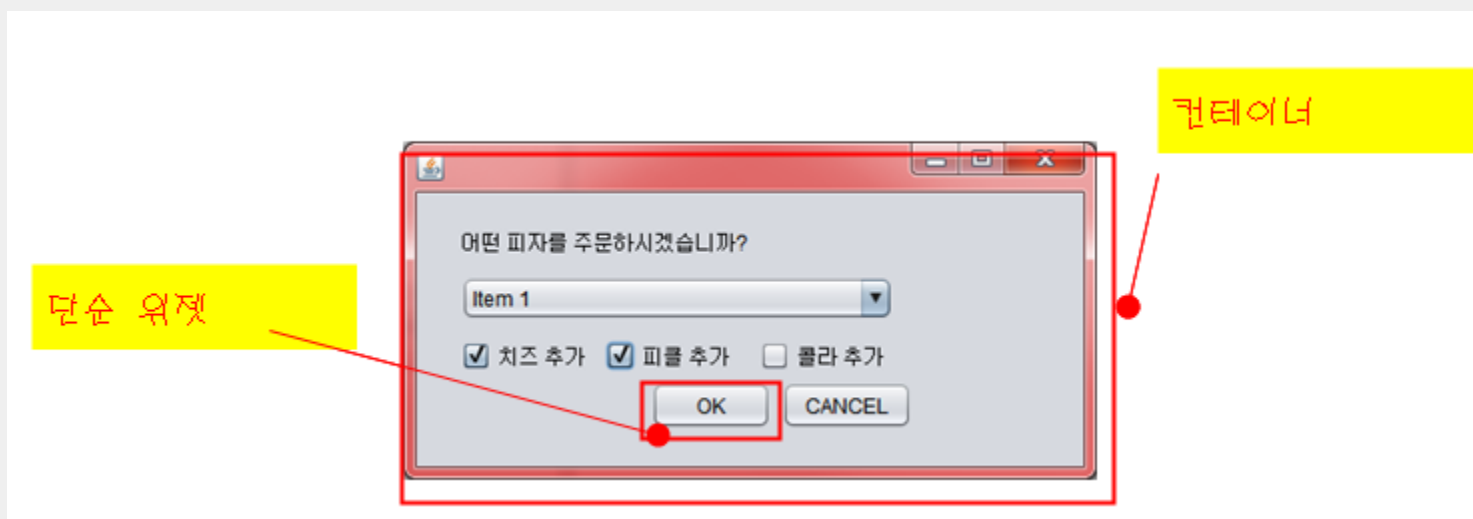
Tkinter 시작하기

- 원도우(window)를 생성하고 여기에 필요한 위젯들을 추가한다.

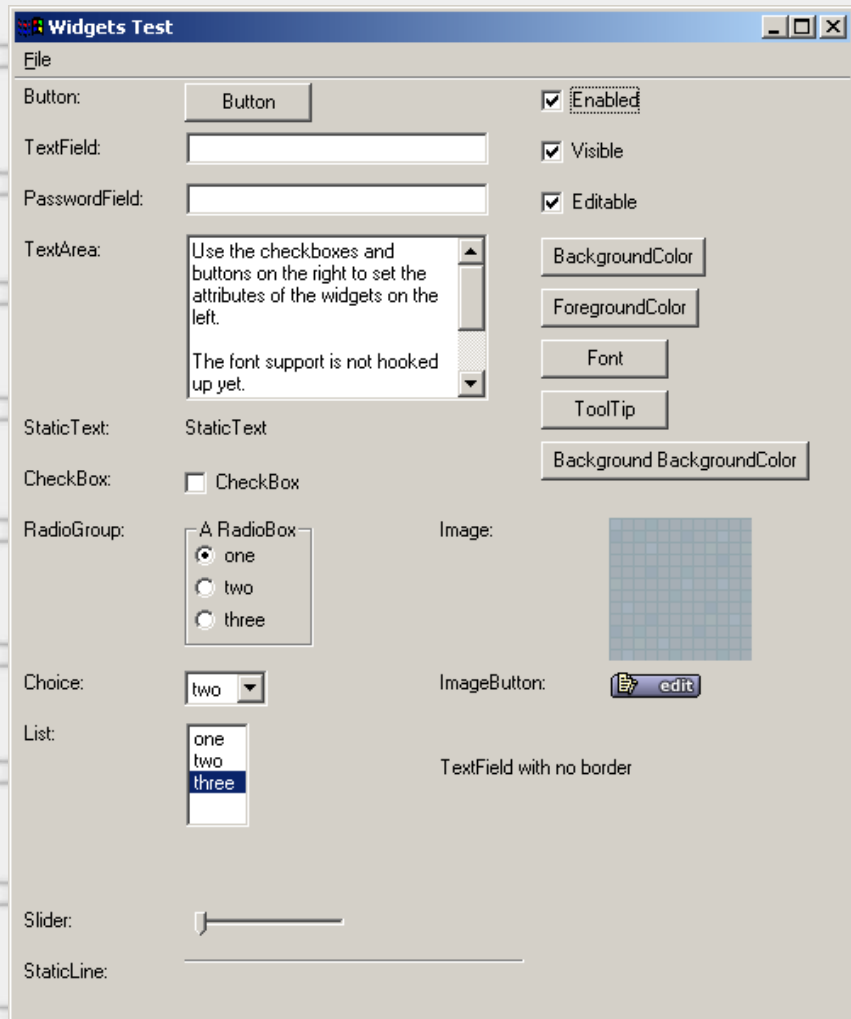


단순 위젯과 컨테이너 컴포넌트

- 단순 위젯: Button, Canvas, Checkbutton, Entry, Label, Message 등이 여기에 속한다.
- 컨테이너 컴포넌트: 다른 컴포넌트를 안에 포함할 수 있는 컴포넌트로서 Frame, Toplevel, LabelFrame, PanedWindow 등이 여기에 속한다.



Tkinter의 위젯들



위젯	설명
Button	간단한 버튼으로 명령을 수행할 때 사용된다.
Canvas	화면에 무언가를 그릴 때 사용한다.
Checkbox	2가지의 구별되는 값을 가지는 변수를 표현한다.
Entry	한 줄의 텍스트를 입력받는 필드이다.
Frame	컨테이너 클래스이다. 프레임은 경계선과 배경을 가지고 있다. 다른 위젯들을 그룹핑하는데 사용된다.
Label	텍스트나 이미지를 표시한다.
Listbox	선택 사항을 표시한다.
Menu	메뉴를 표시한다. 풀다운 메뉴나 팝업 메뉴가 가능하다.
Menubutton	메뉴 버튼이다. 풀다운 메뉴가 가능하다.
Message	텍스트를 표시한다. 레이블 위젯과 비슷하다. 하지만 자동적으로 주어진 크기로 텍스트를 축소할 수 있다.
Radiobutton	여러 값을 가질 수 있는 변수를 표시한다.
Scale	슬라이더를 끌어서 수치를 입력하는데 사용된다.
Scrollbar	캔버스, 엔트리, 리스트 박스, 텍스트 위젯을 위한 스크롤 바를 제공한다.
Text	형식을 가지는 텍스트를 표시한다. 여러 가지 스타일과 속성으로 텍스트를 표시할 수 있다.
Toplevel	최상위 윈도우로 표시되는 독립적인 컨테이너 위젯이다.
LabelFrame	경계선과 제목을 가지는 프레임 위젯의 변형이다.
PanedWindow	자식 위젯들을 크기조절이 가능한 패널로 관리하는 컨테이너 위젯이다.
Spinbox	특정한 범위에서 값을 선택하는 엔트리 위젯의 변형



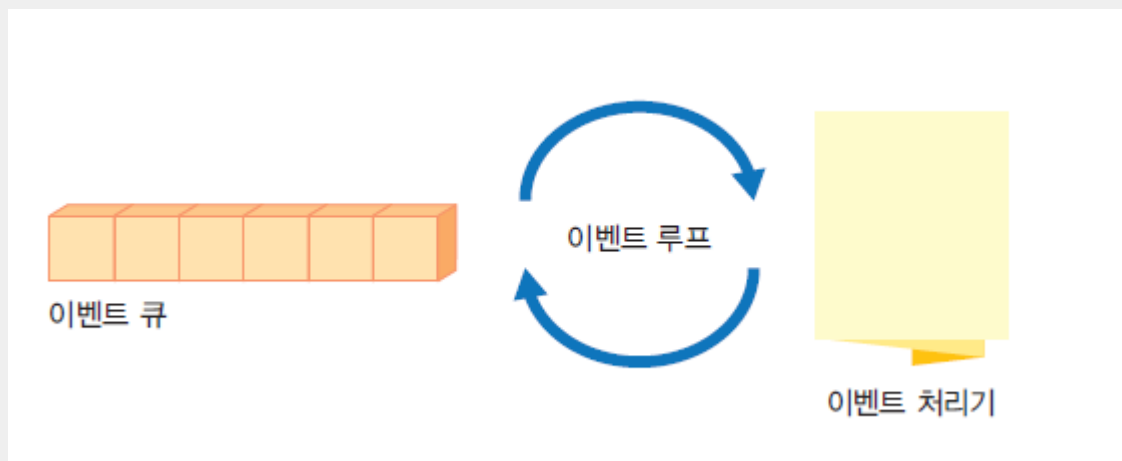
기본윈도우 생성

- 윈도우를 생성해보자

```
from tkinter import *  
window = Tk()  
window.mainloop()
```

분석

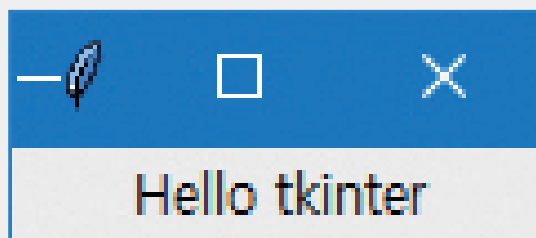
- window.mainloop()
 - 루트 윈도우의 mainloop()는 이벤트 처리 루프로서 사용자로 부터 오는 마우스나 키보드 이벤트를 처리한다



레이블 옵션

- 하나의 레이블이 있는 윈도우를 생성해보자

```
from tkinter import * # tkinter 모듈 가져오기
window = Tk() # 루트 윈도우 생성
label = Label(window, text="Hello tkinter") # 레이블 위젯 생성
label.pack() # 레이블 위젯 배치
window.mainloop() # 윈도우가 종료될 때까지 대기
```



Label 위젯의 옵션

- text** : 글자 자체
- font** : 글꼴과 크기
- fg** : 글자색
- bg** : 배경색
- width** : 위젯의 폭
- height** : 위젯의 높이
- anchor** : 위젯의 위치(n, ne, e, se, s, sw, w, nw, center)
- image** : 그림삽입

버튼 위젯

```
from tkinter import *
```

```
window = Tk()
```

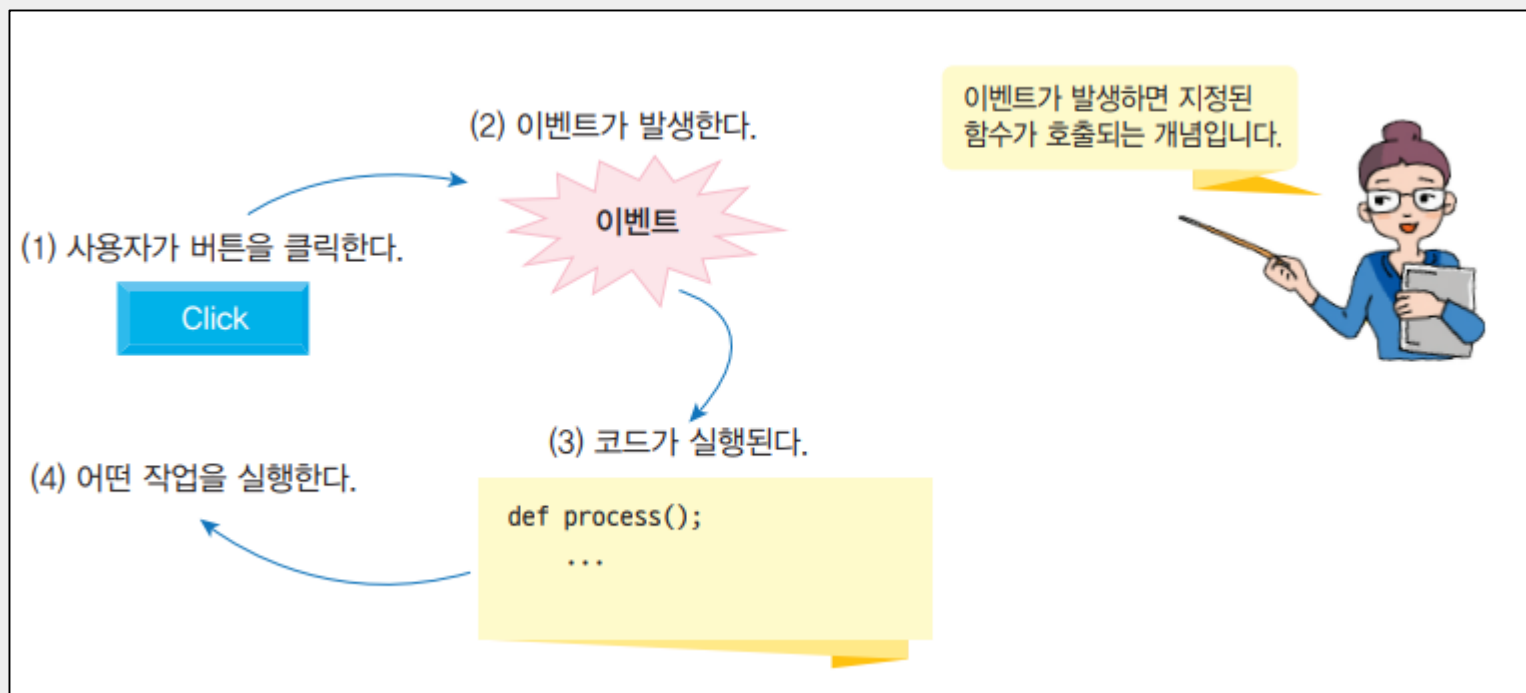
```
button = Button(window, text="클릭 하 세 요!",bg="yellow", fg="blue", width=80,height=2)
```

```
button.pack()
```

```
window.mainloop()
```



버튼 이벤트 처리



버튼 이벤트 처리

Syntax: 버튼 이벤트 처리

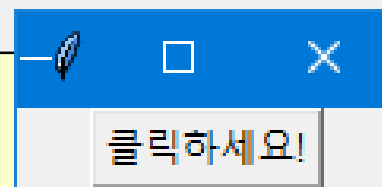
형식 Button = Button(window, command=함수 이름)|

예 Button(window, text="클릭하세요!", command=process)

버튼에서 이벤트가 발생하면
호출되는 함수 등록

예제

```
from tkinter import *  
  
def process():  
    print("버튼이 클릭되었습니다.")  
  
window = Tk()  
button = Button(window, text="클릭하세요!", command=process)  
button.pack()  
  
window.mainloop()
```

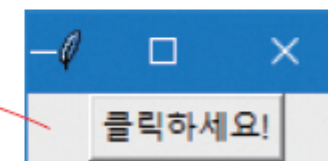


속성을 바꾸는 방법

1. button['text']="누르세요!"
2. button.config(text="누르세요!")

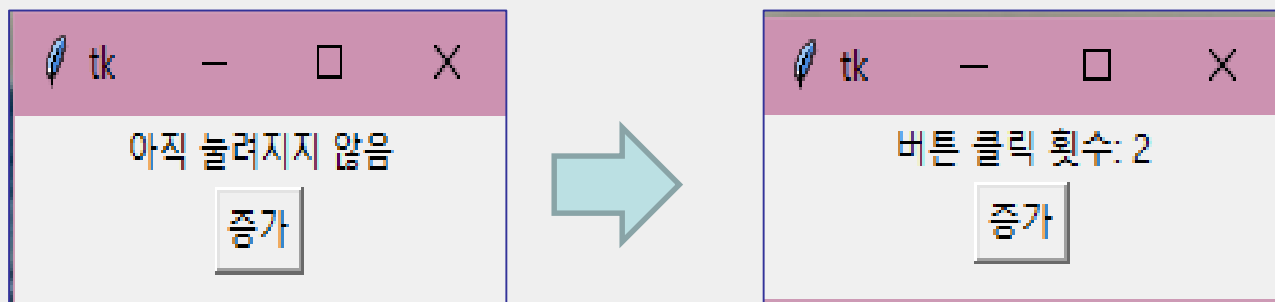
실행결과

```
버튼이 클릭되었습니다.  
버튼이 클릭되었습니다.  
버튼이 클릭되었습니다.  
버튼이 클릭되었습니다.  
버튼이 클릭되었습니다.  
버튼이 클릭되었습니다.
```



실습: 카운터 만들기

- 레이블과 버튼을 사용하여 간단한 카운터를 작성하여 보자. 증가 버튼을 누르면 카운터가 1 증가된다.



속성을 바꾸는 방법 예시

1. `button['text']="누르세요!"`
2. `button.config(text="누르세요!")`

Sol:



인하대학교

```
from tkinter import *

window = Tk()

counter = 0

def clicked():
    global counter
    counter += 1
    label['text'] = '버튼 클릭 횟수: ' + str(counter)

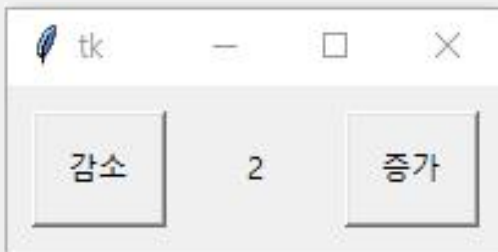
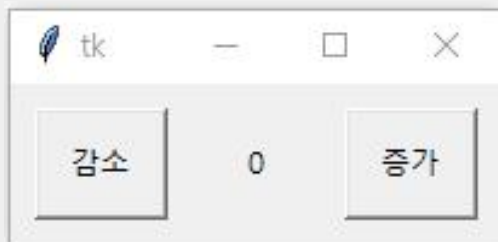
label = Label(window, text="아직 눌러지지 않음")
label.pack()
button = Button(window, text="증가", command=clicked)
button.pack()

window.mainloop()
```

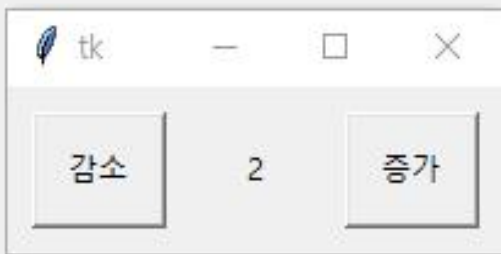
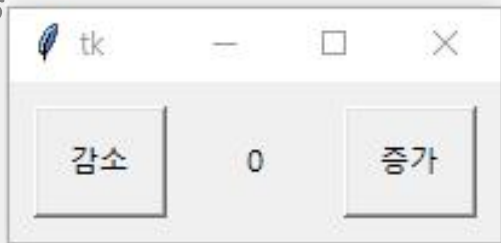
실습 : 증가/감소 버튼

- ‘증가’버튼을 클릭하면 레이블의 값이 1씩 증가
- ‘감소’버튼을 클릭하면 레이블의 값이 1씩 감소

실행결과



코딩



```
from tkinter import *
```

```
window = Tk()
```

```
counter = 0
```

```
def clicked1():  
    global counter  
    counter = counter - 1  
    label.config(text= str(counter))
```

```
def clicked2():  
    global counter  
    counter = counter + 1  
    label.config(text= str(counter))
```

```
button1 = Button(window, text="감 소", command=clicked1)  
button1.pack(side=LEFT, padx=10, pady=10, ipadx=10, ipady=10)
```

```
label = Label(window, text="0")  
label.pack(side=LEFT, padx=10, pady=10, ipadx=10, ipady=10)
```

```
button2 = Button(window, text="증 가", command=clicked2)  
button2.pack(side=LEFT, padx=10, pady=10, ipadx=10, ipady=10)
```

```
window.mainloop()
```

엔트리 위젯



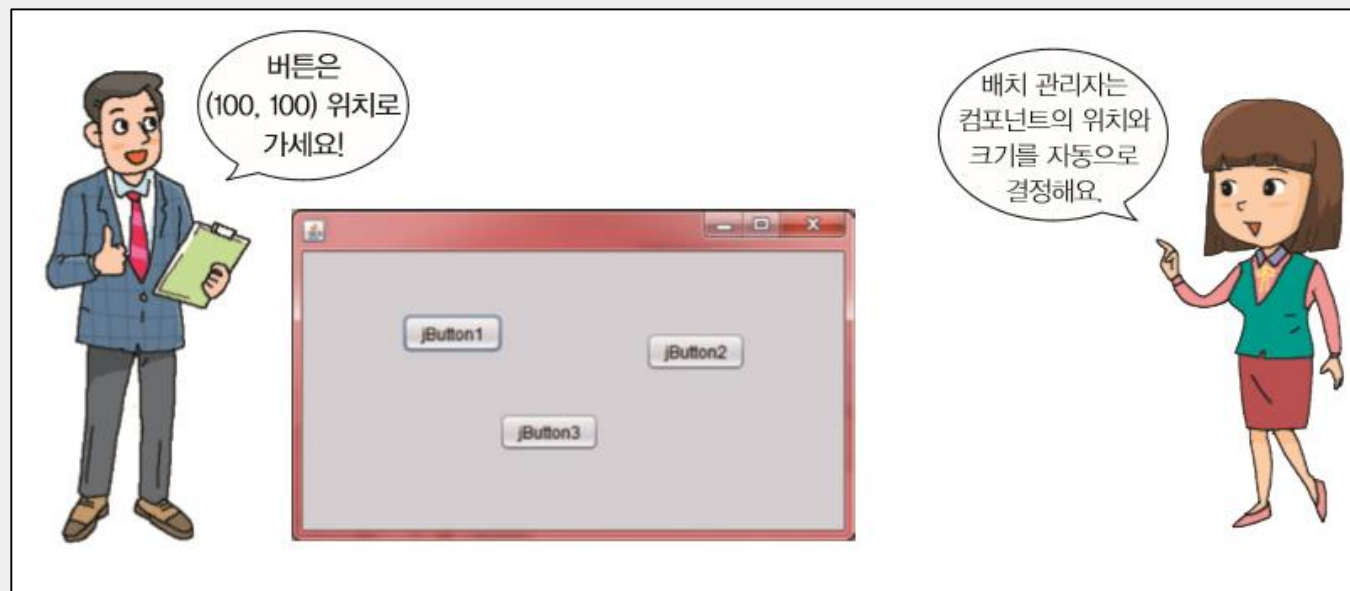
인하대학교

```
from tkinter import *  
window = Tk()  
  
entry = Entry(window, fg="black", bg="yellow", width=80)  
  
entry.pack()  
window.mainloop()
```



배치 관리자

- 압축(pack) 배치 관리자
- 격자(grid) 배치 관리자
- 절대(place) 배치 관리자



1. 압축 배치 관리자

- 압축 배치 관리자(pack geometry manager)는 위젯을 최대한 압축하여 상하로 배치.

```
from tkinter import *  
  
window = Tk()  
  
Label(window, text="박스 #1", bg="red", fg="white").pack()  
Label(window, text="박스 #2", bg="green", fg="black").pack()  
Label(window, text="박스 #3", bg="blue", fg="white").pack()  
  
window.mainloop()
```



=

```
from tkinter import *  
  
window = Tk()  
  
lb1=Label(window, text="박스 #1", bg="red", fg="white")  
lb2=Label(window, text="박스 #2", bg="green", fg="black")  
lb3=Label(window, text="박스 #3", bg="blue", fg="white")  
lb1.pack()  
lb2.pack()  
lb3.pack()  
window.mainloop()
```


압축 배치 관리자

```
from Tkinter import *
```

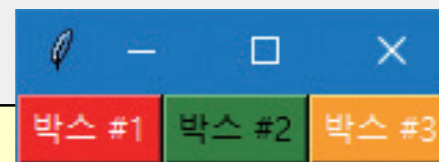
```
window = Tk()
```

```
Button(window, text="박스 #1", bg="red", fg="white").pack(side=LEFT)
```

```
Button(window, text="박스 #2", bg="green", fg="black").pack(side=LEFT)
```

```
Button(window, text="박스 #3", bg="orange", fg="white").pack(side=LEFT)
```

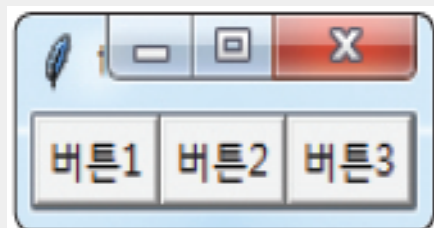
```
window.mainloop()
```



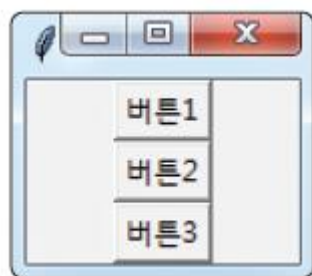
압축배치관리자

- pack() 함수의 옵션 중 수평/수직으로 정렬하는 방법으로 수평 :LEFT,RIGHT 수직:TOP, BOTTOM 사용
 - side = LEFT/RIGHT 옵션은 왼쪽/오른쪽부터 채우라는 의미
 - side = TOP/BOTTOM 옵션은 위/아래부터 채우라는 의미

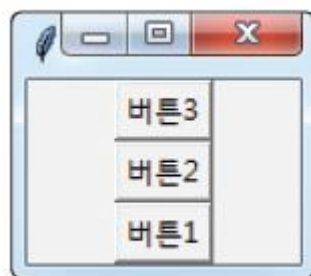
LEFT



TOP



BOTTOM



```
1 from tkinter import *
2 window=Tk()
3
4 button1=Button(window, text="버튼1")
5 button2=Button(window, text="버튼2")
6 button3=Button(window, text="버튼3")
7
8 button1.pack(side=LEFT)
9 button2.pack(side=LEFT)
10 button3.pack(side=LEFT)
11
12 window.mainloop()
```

2. 격자 배치 관리자

	Column 0	Column 1	Column 2
Row 0			
Row 1			
Row 2			
Row 3			

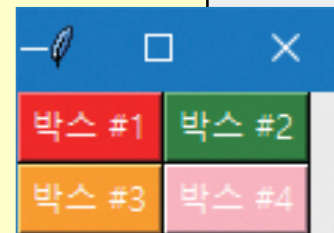
행번호와 열 번호는 0부터 시작합니다.



격자 배치 관리자

- 격자 배치 관리자(grid geometry manager)는 위젯 (버튼, 레이블 등)을 테이블 형태로 배치한다.

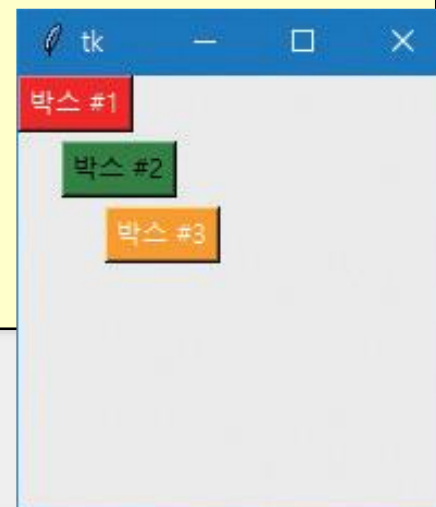
```
from tkinter import *  
window = Tk()  
  
b1 = Button(window, text="박스 #1", bg="red", fg="white")  
b2 = Button(window, text="박스 #2", bg="green", fg="white")  
b3 = Button(window, text="박스 #3", bg="orange", fg="white")  
b4 = Button(window, text="박스 #4", bg="pink", fg="white")  
  
b1.grid(row=0, column=0)      # 0행 0열  
b2.grid(row=0, column=1)      # 0행 1열  
b3.grid(row=1, column=0)      # 1행 0열  
b4.grid(row=1, column=1)      # 1행 1열  
  
window.mainloop()
```



3. 절대 위치 배치 관리자

- 절대위치 배치 관리자(place geometry manager)는 절대위치를 사용하여 위젯을 배치

```
from tkinter import *  
  
window = Tk()  
  
b1 = Button(window, text="박스 #1", bg="red", fg="white")  
b1.place(x=0, y=0)  
b2 = Button(window, text="박스 #2", bg="green", fg="black")  
b2.place(x=20, y=30)  
b3 = Button(window, text="박스 #3", bg="orange", fg="white")  
b3.place(x=40, y=60)  
  
window.mainloop()
```



윈도우의 크기와 위젯의 크기 설정하기

```
from tkinter import *  
  
window = Tk()  
window.geometry("600x100")           # Width x Height, 픽셀  
  
Button(window, text="박스 #1", width=10, height=1).pack()  #글자단위  
Button(window, text="박스 #2", width=10, height=1).pack()  
Button(window, text="박스 #3", width=10, height=1).pack()  
  
window.mainloop()
```



실습: 온도 변환기

```
from tkinter import *
```

```
window = Tk()
```

```
Label(window , text="화씨").grid(row=0, column=0)
```

```
Label(window, text="섭씨").grid(row=1, column=0)
```

```
e1 = Entry(window)
```

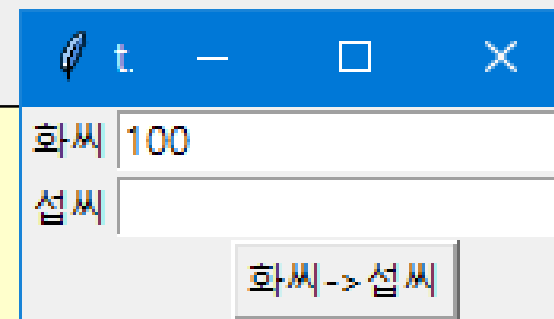
```
e2 = Entry(window)
```

```
e1.grid(row=0, column=1)
```

```
e2.grid(row=1, column=1)
```

```
Button(window, text="화씨->섭씨").grid(row=2, column=1)
```

```
window.mainloop()
```



```
from tkinter import *
```

```
def process():
    temperature = float(e1.get())
    mytemp = (temperature-32)*5/9
    e2.insert(0, str(mytemp))
```

```
window = Tk()
```

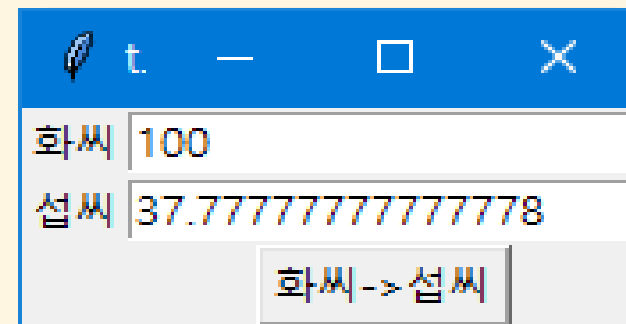
```
l1 = Label(window , text="화씨")
l2 = Label(window, text="섭씨")
l1.grid(row=0, column=0)
l2.grid(row=1, column=0)
```

```
e1 = Entry(window)
e2 = Entry(window)
e1.grid(row=0, column=1)
e2.grid(row=1, column=1)
```

```
b1 = Button(window, text="화씨->섭씨", command=process)
```

```
b1.grid(row=2,column=0, columnspan=2)
```

```
window.mainloop()
```



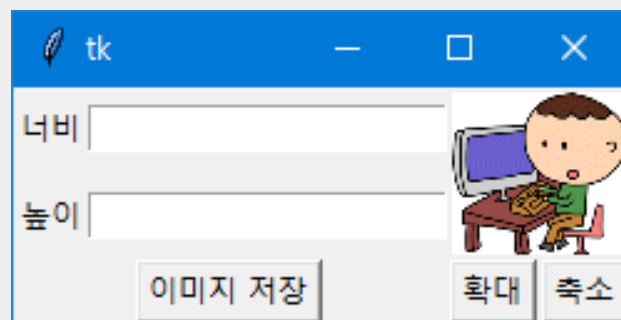
Entry 위젯

- 읽기 : get()
 - 추가 : insert(0,'내용')
 - 삭제 : delete(0,END)
- 엔트리위젯은 문자로만 삽입

실습: 격자 배치 관리자 실습

- 다음과 같이 위젯들을 배치하려고 한다

<label 1>	<entry 1>	<image>	
<label 2>	<entry 2>		
<button 1>		<button 2>	<button 3>



```
from tkinter import *
```

```
window = Tk()
```

```
Label(window, text="너비").grid(row=0, column=0)
```

```
Label(window, text="높이").grid(row=1, column=0)
```

```
e1 = Entry(window)
```

```
e2 = Entry(window)
```

```
e1.grid(row=0, column=1)
```

```
e2.grid(row=1, column=1)
```

```
photo = PhotoImage(file="c:\py_test\fig3.png")
```

```
label = Label(window, image=photo)
```

```
label.grid(row=0, column=2, columnspan=2, rowspan=2)
```

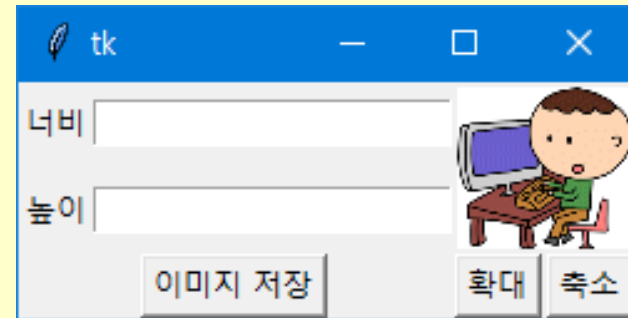
```
Button(window, text='이미지 저장').grid(row=2, column=0, columnspan=2)
```

```
# 두 번째 버튼과 세 번째 버튼은 2열과 3열에 표시한다.
```

```
Button(window, text='확대').grid(row=2, column=2)
```

```
Button(window, text='축소').grid(row=2, column=3)
```

```
window.mainloop( )
```



실습 : 숫자 추측 게임

- 사용자가 컴퓨터가 생성한 숫자(1부터 100사이의 난수)를 알아맞히는 게임을 그래픽 사용자 인터페이스를 사용하여 제작해보자.

실행결과



The image shows two screenshots of a number guessing game's graphical user interface (GUI). Both windows have a blue title bar with the text "숫자를 맞춰보세요!" (Guess the number!) and standard window controls (minimize, maximize, close). The main area of the window contains the text "숫자 게임에 오신 것을 환영합니다!" (Welcome to the number game!).

The top screenshot shows the initial state of the game. Below the welcome message, there is a text input field. To the right of the input field, there are two buttons: "시도" (Attempt) in green and "초기화" (Reset) in red. To the right of these buttons, the text "1부터 100사이의 숫자를 입력하십시오." (Enter a number between 1 and 100) is displayed.

The bottom screenshot shows the state after a user has entered the number "67". The input field now contains "67". The "시도" (Attempt) button is highlighted, and the "초기화" (Reset) button is still visible. To the right of the buttons, the text "낮음!" (Too low!) is displayed, indicating that the user's guess is incorrect and below the target number.

코딩:

1

```
from tkinter import *
import random

answer = random.randint(1,100)

def guessing():
    guess = int(guessField.get())

    if guess > answer:
        msg = "높음!"
    elif guess < answer:
        msg = "낮음!"
    else:
        msg = "정답!"

    resultLabel["text"] = msg
    guessField.delete(0, END)
```

2

```
def reset(): # 정답을 다시 설정한다.
    global answer
    answer = random.randint(1,100)
    resultLabel["text"] = "다시 한번 하세요!"
```

```
window = Tk()
window.title("숫자를 맞춰보세요!")

window.geometry("500x80")

titleLabel = Label(window, text="숫자 게임에 오신 것을 환영합니다!",bg="white")
titleLabel.pack()
```



3

```
guessField = Entry(window)
guessField.pack(side="left")
tryButton = Button(window, text="시도", fg="green", bg="white", command=guessing )
tryButton.pack(side="left")

resetButton = Button(window, text="초기화", fg="red", bg="white", command=reset)
resetButton.pack(side="left")
resultLabel = Label(window, text="1부터 100사이의 숫자를 입력하시오.", bg="white")
resultLabel.pack(side="left")

window.mainloop()
```

실행결과



키보드와 마우스 이벤트 처리

Syntax: tkinter 이벤트 처리

형식 위젯.bind(이벤트지정자 , 콜백함수)

예 frame.bind("<Button-1>", callback)

첫번째 마우스 버튼이 눌리면
callback 함수가 호출된다

마우스 이벤트

- 마우스 이벤트가 처리되는 형식

위젯.bind("마우스이벤트" , 이벤트처리함수)

messagebox.showinfo()
messagebox.showerror()
messagebox.askokcancel()

- 마우스 왼쪽 버튼 클릭

```
1 from tkinter import *
2 from tkinter import messagebox
3 # 함수 정의 부분
4 def clickLeft(event) :
5     messagebox.showinfo("마우스", "마우스 왼쪽 버튼이 클릭됨")
6
7 # 메인 코드 부분
8 window = Tk()
9
10 window.bind("<Button-1>", clickLeft)
11
12 window.mainloop()
```



이벤트 지정자



- 키보드 및 마우스를 누르는 것을 이벤트(Event)라 함.

마우스 작동	관련 마우스 버튼	이벤트 코드
클릭할 때	모든 버튼 공통	〈Button〉
	왼쪽 버튼	〈Button-1〉
	가운데 버튼	〈Button-2〉
	오른쪽 버튼	〈Button-3〉
떼었을 때	모든 버튼 공통	〈ButtonRelease〉
	왼쪽 버튼	〈ButtonRelease-1〉
	가운데 버튼	〈ButtonRelease-2〉
	오른쪽 버튼	〈ButtonRelease-3〉
더블클릭할 때	모든 버튼 공통	〈Double-Button〉
	왼쪽 버튼	〈Double-Button-1〉
	가운데 버튼	〈Double-Button-2〉
	오른쪽 버튼	〈Double-Button-3〉
드래그할 때	왼쪽 버튼	〈B1-Motion〉
	가운데 버튼	〈B2-Motion〉
	오른쪽 버튼	〈B3-Motion〉

마우스 작동	이벤트 코드
마우스 커서가 위젯 위로 올라왔을 때	〈Enter〉
마우스 커서가 위젯에서 떠났을 때	〈Leave〉

키보드 작동	이벤트 코드
모든 키를 누를 때	〈Key〉
특수 키를 누를 때	〈Return〉 〈BackSpace〉 〈Tab〉 〈Shift_L〉 〈Control_L〉 〈Alt_L〉 〈Pause〉 〈Caps_Lock〉 〈Escape〉 〈End〉 〈Home〉 〈Left〉 〈Right〉 〈Up〉 〈Down〉 〈Num_Lock〉 〈Delete〉 〈F1〉~〈F12〉 등
일반 키를 누를 때	a~z A~Z 0~9 〈space〉 〈less〉
화살표 키와 조합	〈Shift-Up〉 〈Shift-Down〉 〈Shift-Left〉 〈Shift-Right〉 등

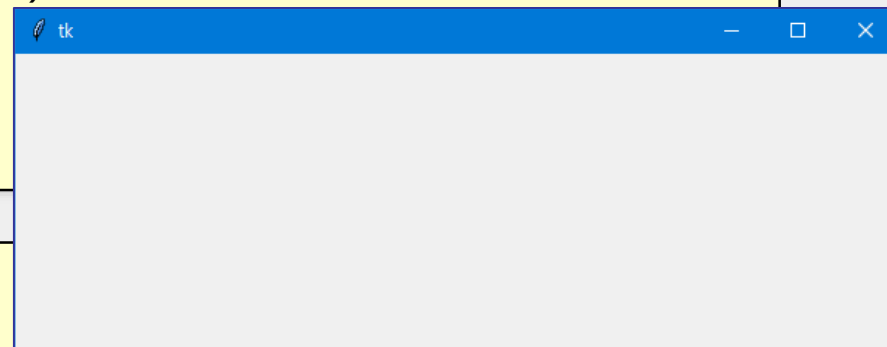
이벤트 처리함수

- 이벤트 처리함수는 모두 `event` 매개변수를 받는다
- 마우스/키보드와 관련된 다양한 정보를 포함
 - 클릭한 위치의 좌표 : `event.x`, `event.y`
 - 마우스 버튼의 번호 : `event.num` - 왼쪽 1, 오른쪽 3
 - 키보드 선택 : `event.keycode`

마우스 이벤트 처리

```
from tkinter import *  
  
window = Tk()  
window.geometry("600x200")  
  
def callback(event):  
    print(event.x, event.y, "에서 마우스 이벤트 발생")  
  
window.bind("<Button-1>", callback)  
window.mainloop()
```

```
32 44 에서 마우스 이벤트 발생  
6 52 에서 마우스 이벤트 발생
```





키보드 이벤트 처리

```
from tkinter import *  
from tkinter import messagebox  
  
window = Tk()  
  
def key_process(event):  
    messagebox.showinfo("키보드 이벤트", "눌린 키 : "+chr(event.keycode))  
  
window.bind("<Key>", key_process)  
  
window.mainloop()
```

chr

chr(i)는 아스키(ASCII) 코드값을 입력으로 받아 그 코드에 해당하는 문자를 출력하는 함수

※ 아스키 코드: 국제 표준 코드체계

: 0에서 127 사이의 숫자들을 각각 하나의 문자 또는 기호에 대응시켜 놓은 것

```
>>> chr(97)
```

```
'a'
```

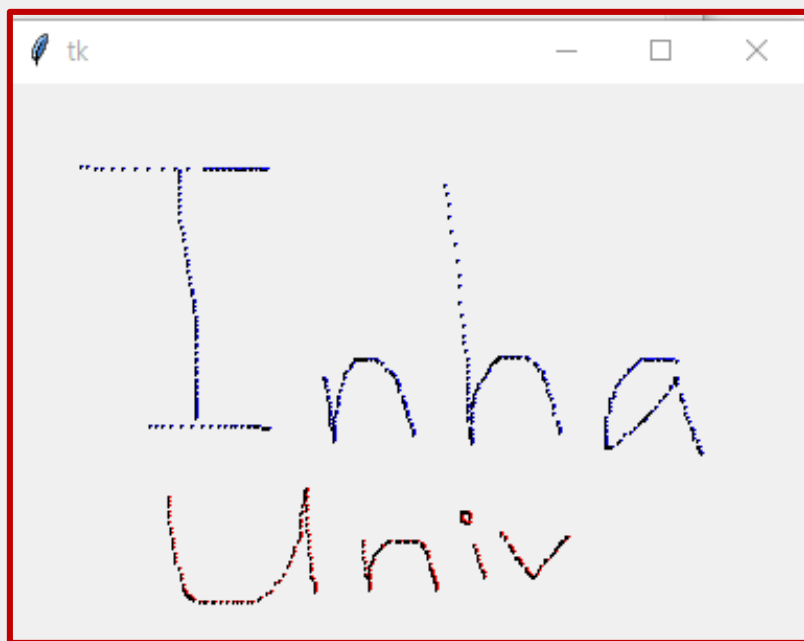
```
>>> chr(48)
```

```
'0'
```

1	0 0 0 0 0 0 0
2	0 0 0 0 0 0 1
3	0 0 0 0 0 1 0
4	0 0 0 0 0 1 1
⋮	⋮
127	1 1 1 1 1 1 0
128	1 1 1 1 1 1 1



실습: 낙서장 만들기



이벤트 지정자



- 키보드 및 마우스를 누르는 것을 이벤트(Event)라 함.

마우스 작동	관련 마우스 버튼	이벤트 코드
클릭할 때	모든 버튼 공통	<Button>
	왼쪽 버튼	<Button-1>
	가운데 버튼	<Button-2>
	오른쪽 버튼	<Button-3>
떼었을 때	모든 버튼 공통	<ButtonRelease>
	왼쪽 버튼	<ButtonRelease-1>
	가운데 버튼	<ButtonRelease-2>
	오른쪽 버튼	<ButtonRelease-3>
더블클릭할 때	모든 버튼 공통	<Double-Button>
	왼쪽 버튼	<Double-Button-1>
	가운데 버튼	<Double-Button-2>
	오른쪽 버튼	<Double-Button-3>
드래그할 때	왼쪽 버튼	<B1-Motion>
	가운데 버튼	<B2-Motion>
	오른쪽 버튼	<B3-Motion>

마우스 작동	이벤트 코드
마우스 커서가 위젯 위로 올라왔을 때	<Enter>
마우스 커서가 위젯에서 떠났을 때	<Leave>

키보드 작동	이벤트 코드
모든 키를 누를 때	<Key>
특수 키를 누를 때	<Return> <BackSpace> <Tab> <Shift_L> <Control_L> <Alt_L> <Pause> <Caps_Lock> <Escape> <End> <Home> <Left> <Right> <Up> <Down> <Num_Lock> <Delete> <F1>~<F12> 등
일반 키를 누를 때	a~z A~Z 0~9 <space> <less>
화살표 키와 조합	<Shift-Up> <Shift-Down> <Shift-Left> <Shift-Right> 등

Canvas 위젯

원호그리기 : `create_arc(x0,y0,x1,y1,option)`

(x1, y1)에서 (x2, y2)의 크기를 가지며 start 각도부터 extent의 각을 지나는 호 생성

```
arc = canvas.create_arc(10, 50, 240, 210, start=0, extent=150, fill= " blue " )
```

a=10,50,240,210

arc=canvas.arc(a, start=.....)

이미지표시 : `create_image(x,y,image,option)` : (x, y) 위치에 image 생성

```
filename = PhotoImage(file = " sunshine.gif " )
```

```
im = canvas.create_image(50, 50, image=filename)
```

선그리기 : `create_line(x0,y0,x1,y1,x2,y2,...xn,yn,option)`

(x0, y0), (x1, y1), ..., (xn, yn) 까지 연결되는 선 생성

```
line = canvas.create_line(x0, y0, x1, y1, ..., xn, yn, options)
```

Canvas 위젯

원이나 타원그리기 : `create_oval(x0,y0,x1,y1,option)`

(x_0, y_0) 에서 (x_1, y_1) 의 크기를 가지는 원 생성

`oval = canvas.create_oval(x0, y0, x1, y1, options)`

다각형그리기 : `create_polygon(x0,y0,x1,y1,x2,y2,...xn,yn,option)`

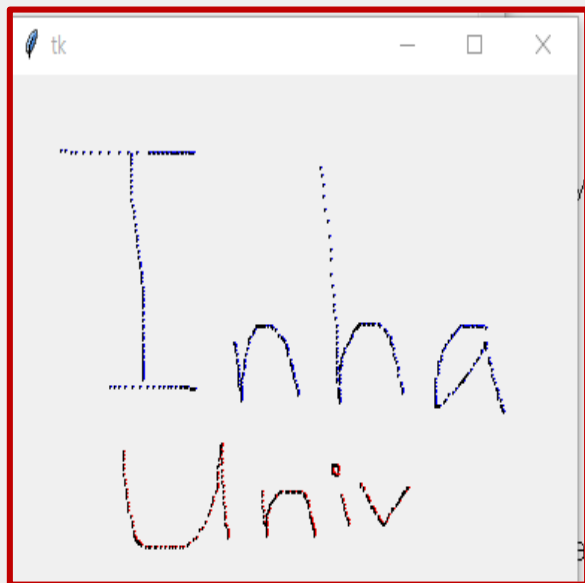
$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ 의 꼭지점을 갖는 다각형 생성

`polygon = canvas.create_polygon(x0, y0, x1, y1, ..., xn, yn, options)`

사각형그리기 : `create_rectangle(x0,y0,x1,y1,option)`

x_0, y_0 : 왼쪽 위 좌표, x_1, y_1 : 오른쪽 아래 좌표

글자그리기 : `create_text(x, y, text, font, option)`



```
from tkinter import *
```

```
def paint(event):
```

```
    x1, y1 = event.x, event.y
```

```
    x2, y2 = event.x+1, event.y+1
```

```
    canvas.create_oval(x1, y1, x2, y2, fill = "blue")
```

```
window = Tk()
```

```
canvas = Canvas(window)
```

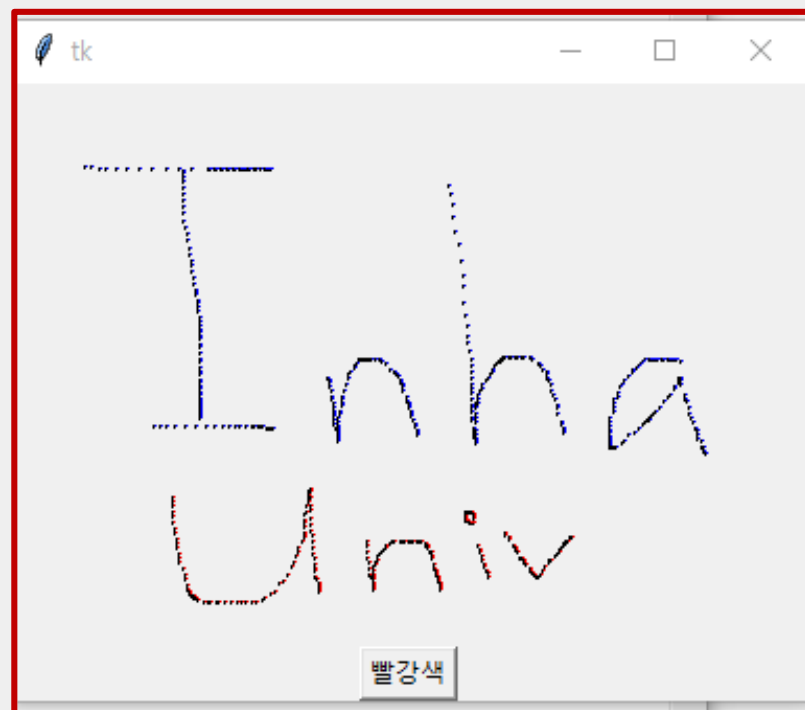
```
canvas.pack()
```

```
canvas.bind("<B1-Motion>", paint)
```

```
window.mainloop()
```



업그레이드: 낙서장 만들기





```
from tkinter import *
```

```
mycolor = "blue"
```

```
def paint(event):
```

```
    x1, y1 = event.x, event.y
```

```
    x2, y2 = event.x+1, event.y+1
```

```
    canvas.create_oval(x1, y1, x2, y2, fill = mycolor)
```

```
def change_color():
```

```
    global mycolor
```

```
    mycolor="red"
```

```
window = Tk()
```

```
canvas = Canvas(window)
```

```
canvas.pack()
```

```
canvas.bind("<B1-Motion>", paint)
```

```
button = Button(window, text="빨강색", command=change_color)
```

```
button.pack()
```

```
window.mainloop()
```